LangChain4j with Spring Boot

Author: Konrad Kowalczyk
Repository: xkondix AI Sandbox

Presentation Plan

1. Introduction
2. Configuration
3. Integration with LLMs
4. Chat
5. Chat with memory
6. Retrieval-Augmented Generation (RAG)
7. Tools (Function Calling)
8. Model Context Protocol (MCP)
9. Questions



Introduction

- A **Large Language Model (LLM)** is an advanced AI model trained on massive amounts of text data to understand and generate human-like language. It can answer **questions, write code, summarize documents, translate languages** and perform many other language-related tasks.
- **Token** is a basic unit of text (like a word or part of a word) that language models use to process and generate responses.
- The purpose of this presentation and project is to demonstrate how you can easily integrate **Spring Boot** with **LLMs** using the **LangChain4j** library.

Project Setup

- Spring Boot 3.5.3 (with LangChain4j)
- JAVA 21
- Ollama for local LLMs
- Redis for memory
- ChromaDB for vector store

LangChain4j dependencies

Useful dependencies (more in project):

- langchain4j & langchain4j-core (core)
- langchain4j-open-ai (gpt api)
- langchain4j-ollama (local LLMs)
- langchain4j-embeddings (models converting text -> vectors)
- langchain4j-embeddings-all-minilm-l6-v2-q (model used in project)
- langchain4j-chroma (chromaDB used as a vector database in the project)
- langchain4j-redis (used as a permanent chat memory in the project)

Docker compose

Steps required to download LLMs:

1. `docker compose up -d`
2. `docker exec -it ollama bash`
3. `ollama pull gemma3:4b`
4. `ollama pull llama3.1:8b`




```yaml
version: '3.8'

services:
  ollama: #local llm image container
    image: ollama/ollama
    container_name: ollama
    restart: unless-stopped
    ports:
      - "11434:11434"
    volumes:
      - ollama:/root/.ollama  # keep models between restarts

  redis: #chat memory
    image: redis:latest
    container_name: redis
    ports:
      - "6379:6379"
    restart: unless-stopped

  chroma: #vectore store
    image: ghcr.io/chroma-core/chroma:0.6.4.dev226
    container_name: chroma
    ports:
      - "8000:8000"

volumes:
  ollama:
```

Model Parameters (Local)

- **modelName,** the name of the model (e.g. gemma3:4b)
- **baseUrl,** Ollama server (e.g. http://localhost:11434)
- **temperature,** controls the randomness and creativity of a language model's output
  - Higher = more creatively and unpredictably
  - Lower = more deterministic
- **topP,** it limits the selection to a dynamic number of tokens whose cumulative probability exceeds a specified threshold p (value > p)
- **topK,** It limits the selection of the next token to a strictly defined number k of the most probable options
- **logRequests && logResponses,** enable logging
- **maxRetries,** repeats the query in case of server errors (e.g., 500, timeout)
- **timeout,** sets the maximum waiting time for a response

Model Parameters (OpenAi)

- **apiKey,** key generated in openAi to gain access
- **maxTokens,** the maximum number of output tokens that can be generated in the chat completion



It may be useful, each 1000 tokens has a price

e.g. “gpt-4o-mini” for 1000 tokens

- input tokens 0.00015 $
- output tokens 0.0006 $

```
OpenAiChatModel model = OpenAiChatModel.builder()
        .apiKey(System.getenv("OPENAI_API_KEY"))
        .modelName("gpt-4o-mini")
        .temperature(0.3)
        .timeout(ofSeconds(60))
        .logRequests(true)
        .logResponses(true)
        .build();
```

more below

- LangChain4j
- OpenAi

Local (gemma3)

```java
@Bean("gemmaModel")
public OllamaChatModel gemmaChatModel() {
    return OllamaChatModel.builder()
            .baseUrl("http://localhost:11434")
            .modelName("gemma3:4b")
            .maxRetries(3)
            .timeout(Duration.ofSeconds(120))
            .logRequests(true)
            .logResponses(true)
            .build();
}
```

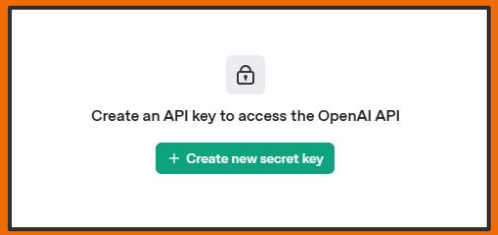
Local (llama 3.1)

```java
@Bean("llamaModel")
public OllamaChatModel llamaChatModel() {
    return OllamaChatModel.builder()
            .baseUrl("http://localhost:11434")
            .modelName("llama3.1:8b")
            .logRequests(true)
            .logResponses(true)
            .build();
}
```

API OpenAI (chat GPT)

Using ChatGPT in your application requires an OpenAI API key and specifying the desired model (like gpt-4) when making requests to the API.

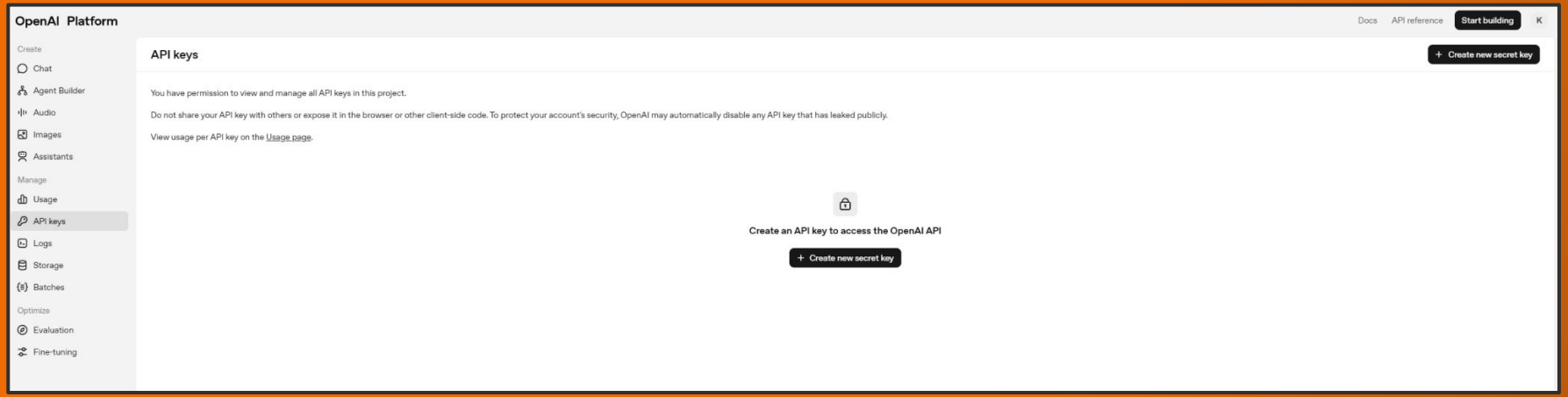
Link to create a key GPT key




```
@Value("${OPENAI_API_KEY}")
String key;

no usages  Konrad
@Bean
public OpenAiChatModel gptChatModel() {
    return OpenAiChatModel.builder()
            .apiKey(key)
            .modelName("gpt-4o")
            .logRequests(true)
            .logResponses(true)
            .build();
}
```

API OpenAI (chat GPT)



Chat

Simple chat without additional features

```java
@Service
public class GemmaService {
    2 usages
    @Qualifier("gemmaModel")
    private final OllamaChatModel gemmaChatModel;

    no usages  Konrad
    public GemmaService(@Qualifier("gemmaModel") OllamaChatModel gemmaChatModel) {
        this.gemmaChatModel = gemmaChatModel;
    }
    Konrad
    public String chat(String message) { return gemmaChatModel.chat(message); }
}
```

AiServices

AI Services in LangChain4j provide a declarative interface to interact with LLMs.

Models, hiding the complexity behind simple, annotated methods.

It supports features such as:

- ➢ Memory
- ➢ Tools
- ➢ RAG

```
public interface Assistant {
    Konrad
    String chat(String message);
}
```

```
this.assistant = AiServices.builder(Assistant.class)
        .chatModel(gemmaChatModel)
        .chatMemory(chatMemory)
        .build();
```

Chat with Temporary Memory

There are two common approaches for temporary memory:

1. with user
2. without user

```
public GemmaMemoryService(@Qualifier("gemmaModel") OllamaChatModel gemmaChatModel) {
    ChatMemory chatMemory = MessageWindowChatMemory.withMaxMessages(10);

    MessageWindowChatMemory.builder()
            .id("user-id")
            .maxMessages(3)
            .build();

    this.assistant = AiServices.builder(Assistant.class)
            .chatModel(gemmaChatModel)
            .chatMemory(chatMemory)
            .build();
}
1 usage  Konrad
public String chatWithHistory(String message) {
    return assistant.chat(message);
}
```

Chat with Permanent Memory (redis)

Redis is an in-memory data store that can be used to **persist chat memory across sessions**.

Before using it, we need to **configure the Redis database** and **integrate it with our application**, as demonstrated in the setup screenshots.

```
@Bean
public ChatMemoryStore redis()
{
    return RedisChatMemoryStore.builder()
            .host("localhost")
            .port(6379)
            .build();
}
```

```
public String chatWithRedis(String userId, String message) {
    ChatMemory memory = TokenWindowChatMemory.builder()
            .chatMemoryStore(redis)
            .id(userId)
            .maxTokens( maxTokens: 1000, new OpenAiTokenCountEstimator(GPT_4_0))
            .build();
    memory.add(userMessage(message));
    ChatResponse response = model.chat(memory.messages());
    log.info("token usage = {}", response.tokenUsage());
    AiMessage aiMessage = response.aiMessage();
    memory.add(aiMessage);
    return aiMessage.text();
}

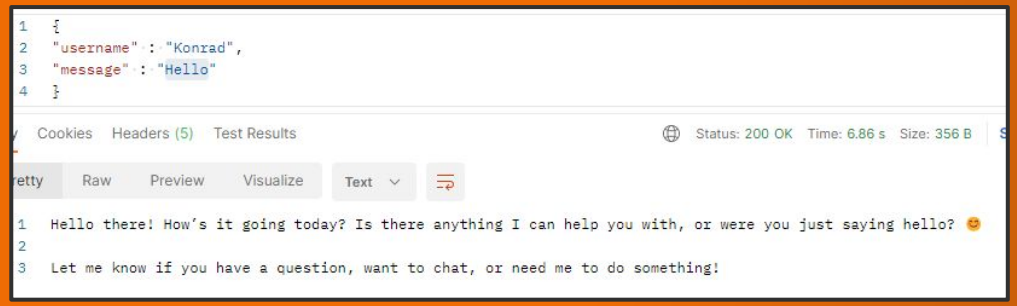
1 usage  Konrad
public String chatWithRedisAiAssistant(String userId, String message) {
    return assistantCache.computeIfAbsent(userId, id -> {
        ChatMemory memory = TokenWindowChatMemory.builder()
                .chatMemoryStore(redis)
                .id(userId)
                .maxTokens( maxTokens: 1000, new OpenAiTokenCountEstimator(GPT_4_0))
                .build();

        return AiServices.builder(Assistant.class)
                .chatModel(model)
                .chatMemory(memory)
                .build();
    }).chat(message);
}
```

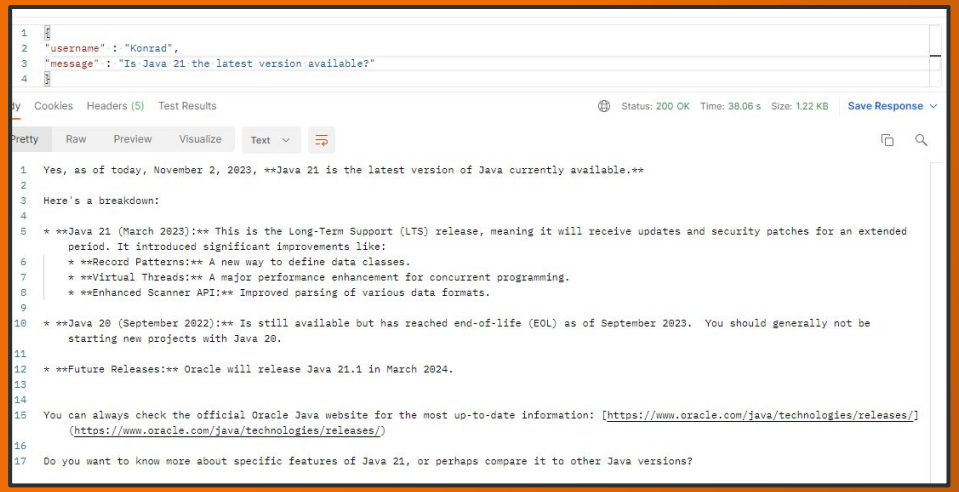
Tokens

Token Usage:

{ inputTokenCount = 10,
outputTokenCount = 50,
totalTokenCount = 60 }



{ inputTokenCount = 79,
outputTokenCount = 285,
totalTokenCount = 364 }



Llama 3 Tokenizer

Retrieval-Augmented Generation (RAG)

RAG = combines LLMs with external knowledge for more accurate answers

- 📄 Document Search
- 🌐 Web Search
- 🗄️ Database Search

Retriever, finds relevant data (from docs, DB, or the web)

Vector Store, stores document embeddings for similarity search

LLM, generates final answer using retrieved context

Documents Search

The provided configuration in the project is adapted specifically for document-based RAG. It uses Chroma vector storage and embedding model to process and retrieve relevant text segments from uploaded files.

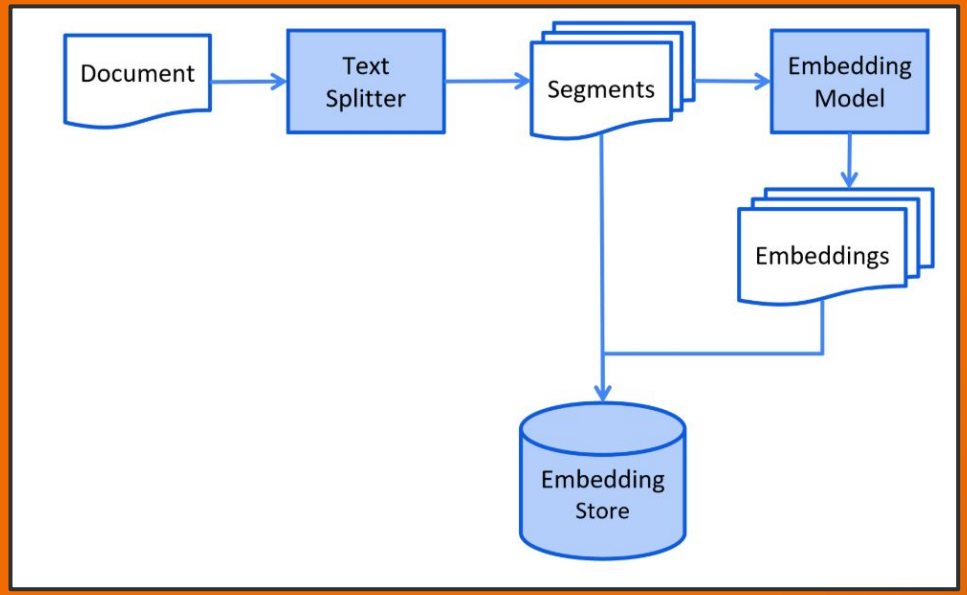
Sources of retrieval:

- 📄 Local documents
- 🗒️ PDF files
- ✍️ Plain text files

```
@Bean
public EmbeddingModel embeddingModel() { return new AllMiniLmL6V2QuantizedEmbeddingModel(); }
```

```
@Bean
public EmbeddingStore<TextSegment> embeddingStore() {
    return ChromaEmbeddingStore.builder()
            .baseUrl("http://localhost:8000")
            .collectionName("documents")
            .build();
}
```

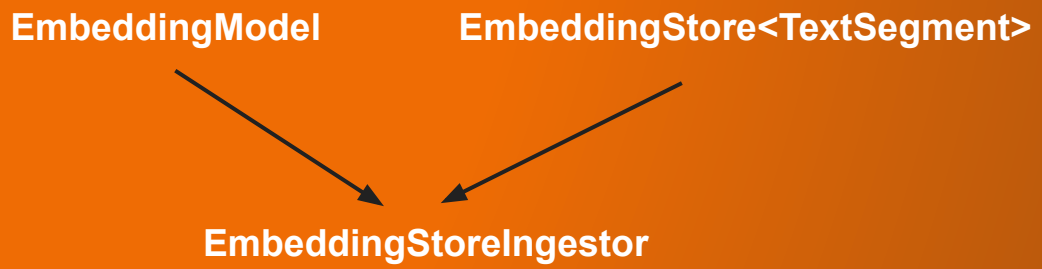
Retrieval-Augmented Generation (RAG)



source: https://docs.langchain4j.dev/tutorials/rag

Documents Search (Vector Database)

- **EmbeddingModel** transforms input text into numeric vector representations (embeddings) used for semantic similarity search in the vector database (e.g., Chroma).
- **EmbeddingStore** provides access to a local Chroma vector database that stores and retrieves text embeddings for semantic search operations.
- **EmbeddingStoreIngestor** splits documents into smaller segments, generates embeddings for each segment, and stores them in the vector database.



Documents Search (Database Score)

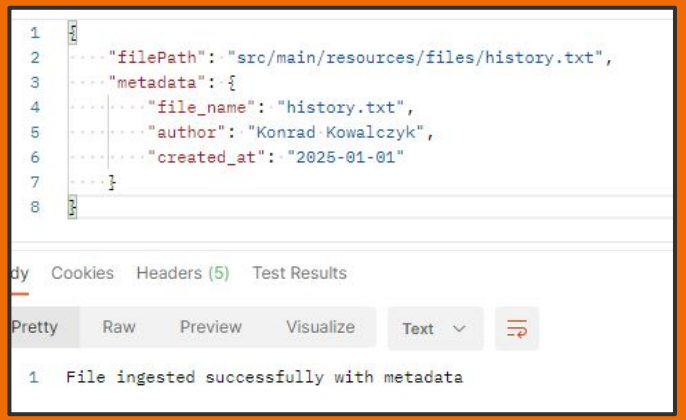
Score represents the similarity measure between the **query embedding** and the **stored embeddings** in the **vector database**.

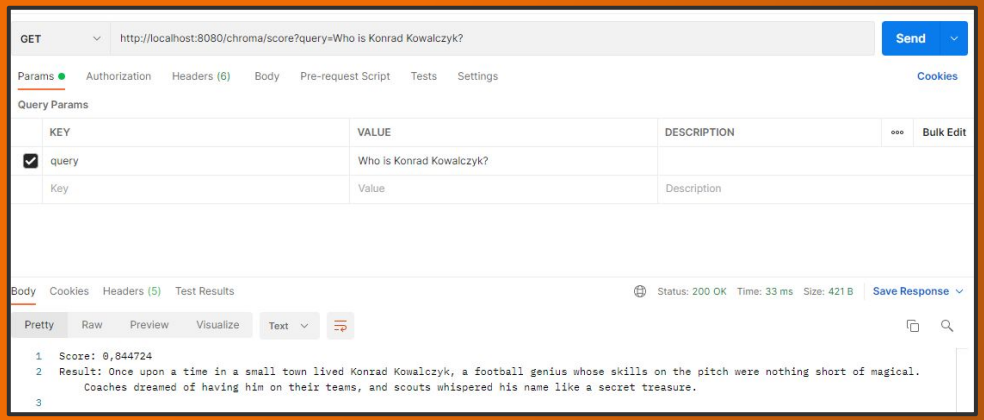
It indicates how closely the retrieved text segment matches the meaning of the input query.

Similarity Score (Chroma), range from 0 to 1

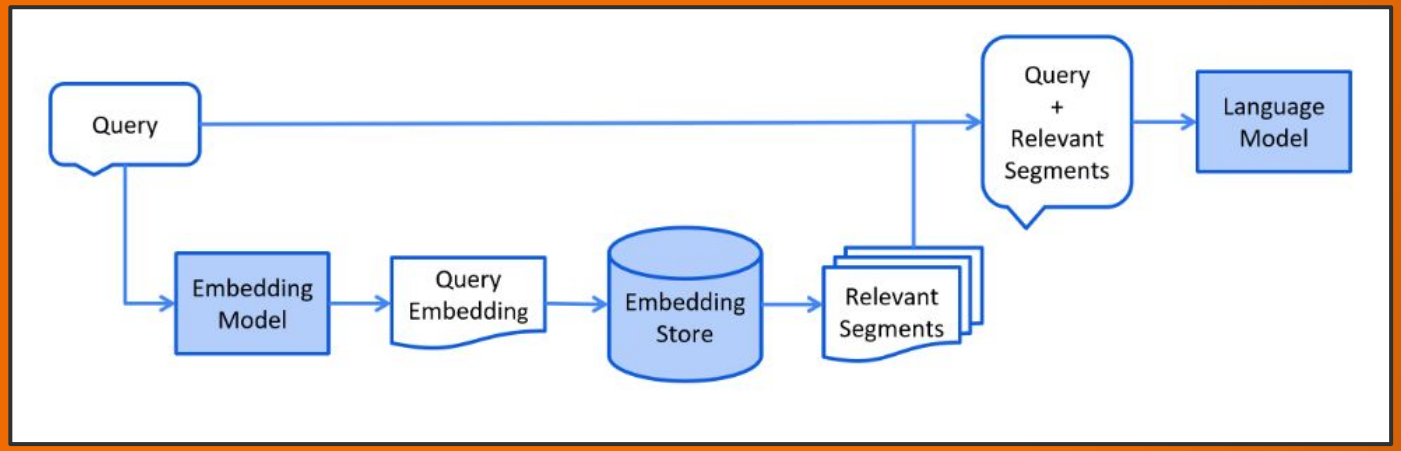
0 → no similarity

1 → perfect match





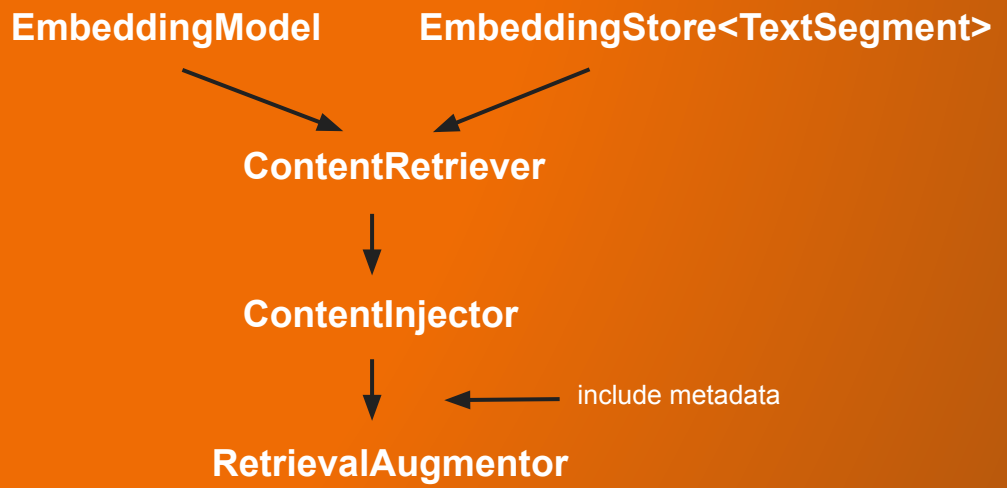
Retrieval-Augmented Generation (RAG)



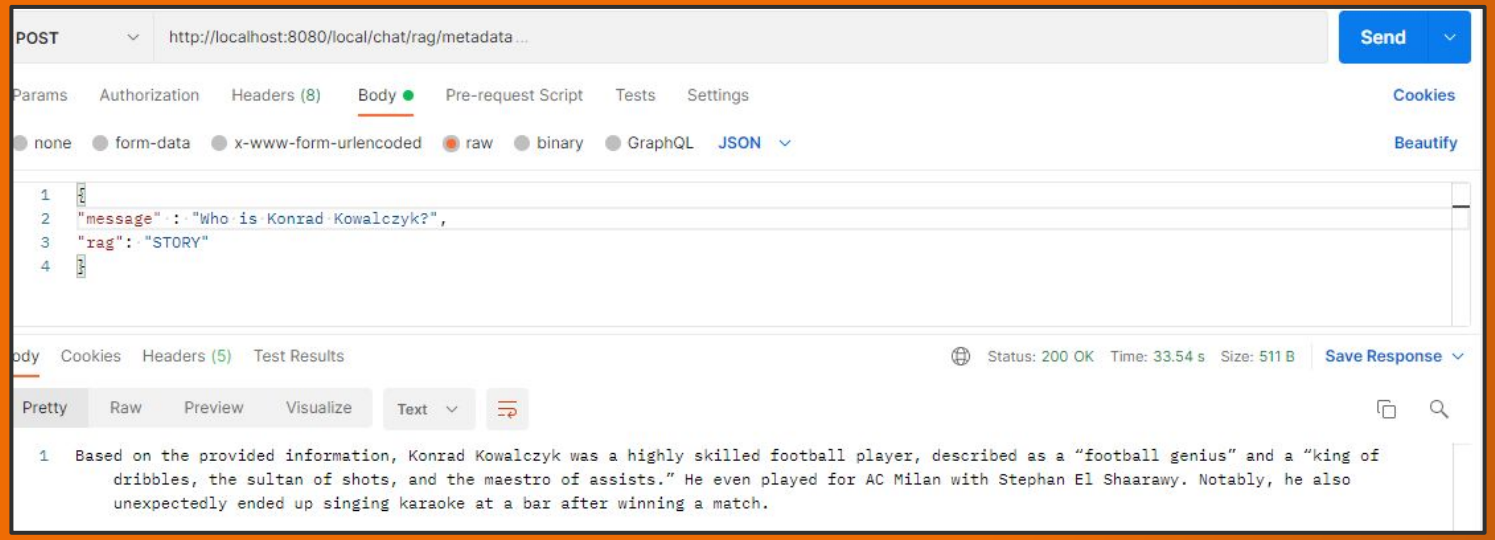
source: https://docs.langchain4j.dev/tutorials/rag

Documents Search (Query)

- **ContentRetriever** retrieves relevant segments from the vector database by embedding user queries and finding the most similar document chunks.
- **ContentInjector** configures which metadata fields (e.g., file name and segment index) should be included in the prompt sent to the LLM during RAG (Retrieval-Augmented Generation).
- **RetrievalAugmentor** combines the retriever and injector to enhance user queries with relevant context before passing them to the LLM.

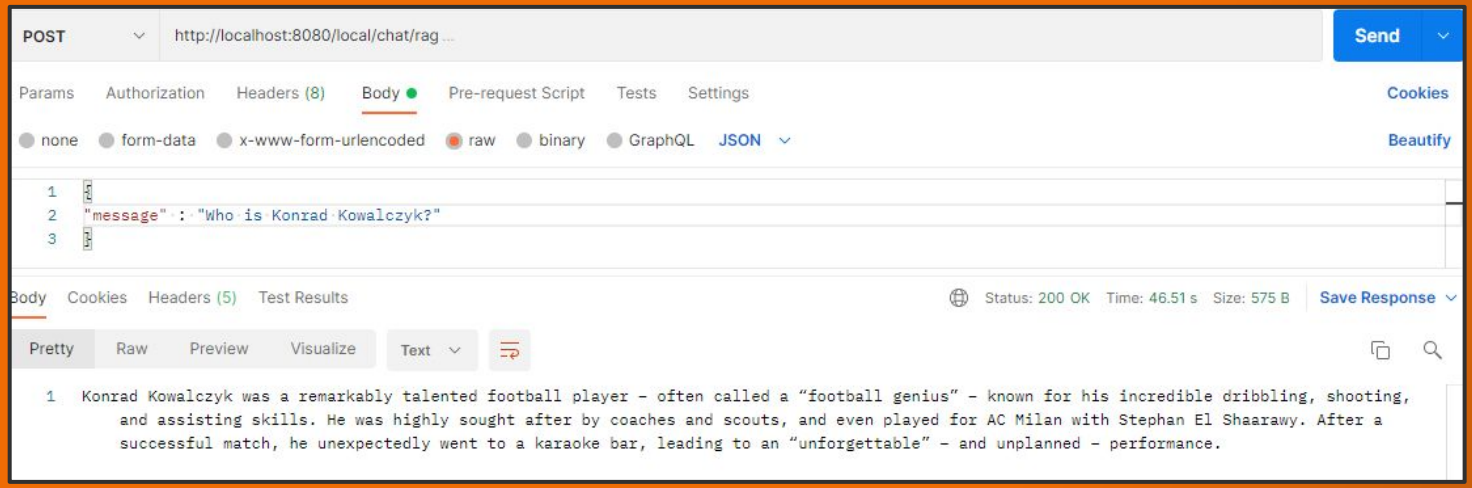


Documents Search (Query with metadata)




```
"content" : "Who is Konrad Kowalczyk?\n\nAnswer using the following information:\ncontent: Once upon a time in a small town lived Konrad Kowalczyk, a footbal
 name like a secret treasure.\nfile_name: history.txt\nindex: 0\nauthor: Konrad Kowalczyk\ncreated_at: 2025-01-01\n\ncontent: Konrad was the king of dribbles
 .\nfile_name: history.txt\nindex: 1\nauthor: Konrad Kowalczyk\ncreated_at: 2025-01-01\n\ncontent: One fateful evening, after winning yet another crucial mat
  was... unforgettable — though not exactly in the way he intended.\nfile_name: history.txt\nindex: 3\nauthor: Konrad Kowalczyk\ncreated_at: 2025-01-01"
],
```

Documents Search (Query without metadata)




```
"content" : "Who is Konrad Kowalczyk?\n\nAnswer using the following information:\nOnce upon a time in a small town lived Konrad Kowalczyk, a football genius whose
  secret treasure.\n\nKonrad was the king of dribbles, the sultan of shots, and the maestro of assists. In fact, he even played for AC Milan alongside the talente
 that his vocal talents could match his football finesse, he took the stage. What followed was... unforgettable — though not exactly in the way he intended."
],
```

Web Search

Before you start, you need to create an account at **Tavily platform** to obtain an API key.

You can also choose a different web search engine, but this one is recommended.

Example: **Web Search Example**

```
1 usage
WebSearchEngine webSearchEngine = TavilyWebSearchEngine.builder()
        .apiKey(System.getenv( name: "TAVILY_API_KEY"))
        .build();

no usages
ContentRetriever webSearchContentRetriever = WebSearchContentRetriever.builder()
        .webSearchEngine(webSearchEngine)
        .maxResults(3)
        .build();
```

Database Search

Before you start, you need to configure and establish a connection to your database.

This approach allows the RAG pipeline to retrieve relevant information directly from relational or NoSQL data sources, for example, from tables containing logs, customer data, or product details.

Example: **Database Search Example**

Tools (function calling)

Tools = methods the LLM uses to perform tasks

Not all models support function calling ❌

```
no usages  Konrad
public LlamaArtistService(@Qualifier("llamaModel") OllamaChatModel llamaChatModel, ArtistToolComponent artistTool) {
    this.assistant = AiServices.builder(ArtistAssistant.class)
            .chatModel(llamaChatModel)
            .tools(artistTool)
            .build();
}

1 usage  Konrad
public String generateStory(String topic, int lines) { return assistant.writeStoryAndSaveToFile(topic, lines); }

1 usage  Konrad
public String generateStoryMessage(String message) { return assistant.writeStoryAboutTopic(message); }
```

@Tool

@Tool is an annotation that marks a method with a description to help the LLM choose the right tool for a given task.

```java
@Tool("Save the given story to a file on disk and return name of the story")
public String saveStoryToFile(String story) {
    String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss"));
    String filename = OUTPUT_DIR + "/story_" + COUNTER.getAndIncrement() + "_" + timestamp + ".txt";

    try (FileWriter writer = new FileWriter(filename)) {
        writer.write(formatTextToNewLines(story));
        log.info("Story saved to file: {}", filename);
        return filename;
    } catch (IOException e) {
        log.error("Failed to write poem to file", e);
        return "Exception occurs";
    }
}
```

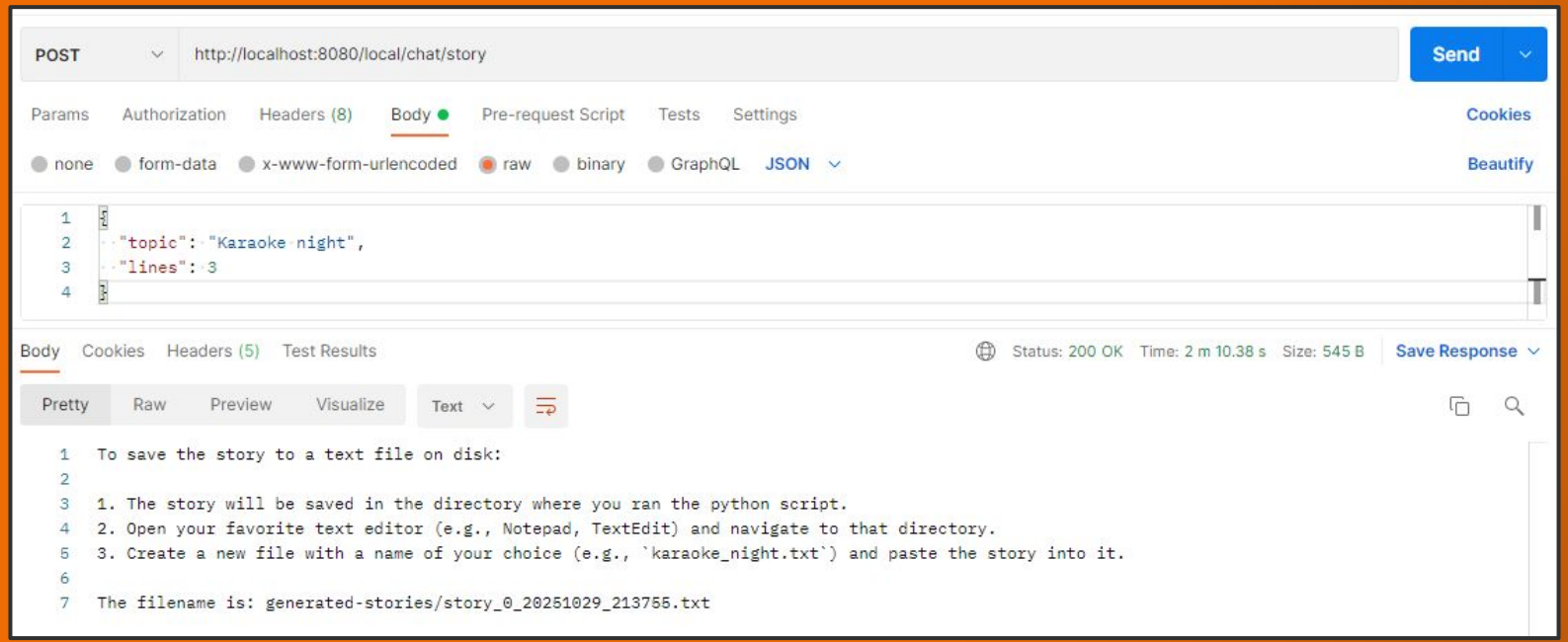
@SystemMessage && @UserMessage

@SystemMessage, defines the **initial system instruction** for the LLM. It sets the model's behavior, role, or personality.

@UserMessage, represents a **message from the user**. What the user inputs or asks during the conversation.

```
@SystemMessage("You are a professional author writing in the style of J.R.R. Tolkien. Your stories should reflect epic fantasy themes, deep world-building, and lyrical language.")
@UserMessage("""
        Write a story about {{topic}}. The story should be {{lines}} lines long and your response should only include the filename.
        Then save this story to a text file on disk.
        """)
String writeStoryAndSaveToFile(@V("topic") String topic, @V("lines") int lines);
```

Example (request)

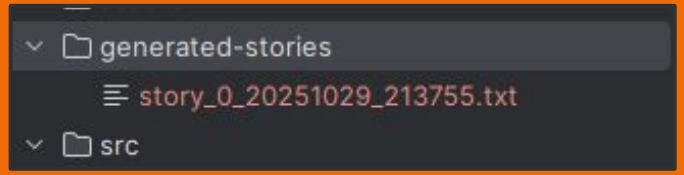


Example (logs)

```
"model" : "llama3.1:8b",
"messages" : [ {
  "role" : "system",
  "content" : "You are a professional author writing in the style of J.R.R. Tolkien. Your stories should reflect epic fantasy themes, deep world-building, and lyrical language."
}, {
  "role" : "user",
  "content" : "Write a story about Karaoke night. The story should be 3 lines long and your response should only include the filename.\nThen save this story to a text file on disk.\n"
} ],
```

```
"tools" : [ {
  "type" : "function",
  "function" : {
    "name" : "saveStoryToFile",
    "description" : "Save the given story to a file on disk and return name of the story",
    "parameters" : {
      "type" : "object",
      "properties" : {
        "story" : {
          "type" : "string"
        }
      },
      "required" : [ "story" ]
    }
  }
} ]
```

Example (result)




```
1  The sun had set over the bustling streets, casting a warm orange glow over the crowded karaoke bar.
2  The air was alive with laughter and music, as patrons took turns belting out their favorite tunes.
3  In this joyful atmosphere, even the most tone-deaf singers found themselves lost in the moment.
```


2025-10-29T21:38:43.981+01:00 INFO 20460 --- [ai] [nio-8080-exec-1] d.l.http.client.log.LoggingHttpClient : HTTP response:
- status code: 200
- headers: [content-length: 690], [content-type: application/json; charset=utf-8], [date: Wed, 29 Oct 2025 20:38:43 GMT]
- body: {"model":"llama3.1:8b","created_at":"2025-10-29T20:38:43.979890332Z","message":{"role":"assistant","content":"To save the story to a text file on disk:\n\n1. The story will be saved in the directory where you ran the python script.\n2. Open your favorite text editor (e.g., Notepad, TextEdit) and navigate to that directory.\n3. Create a new file with a name of your choice (e.g., `karaoke_night.txt`) and paste the story into it.\n\nThe filename is: generated-stories/story_0_20251029_213755.txt"},"done_reason":"stop","done":true,"total_duration":48181538882,"load_duration":42034725,"prompt_eval_count":226,"prompt_eval_duration":22900018756,"eval_count":100,"eval_duration":25232814929}

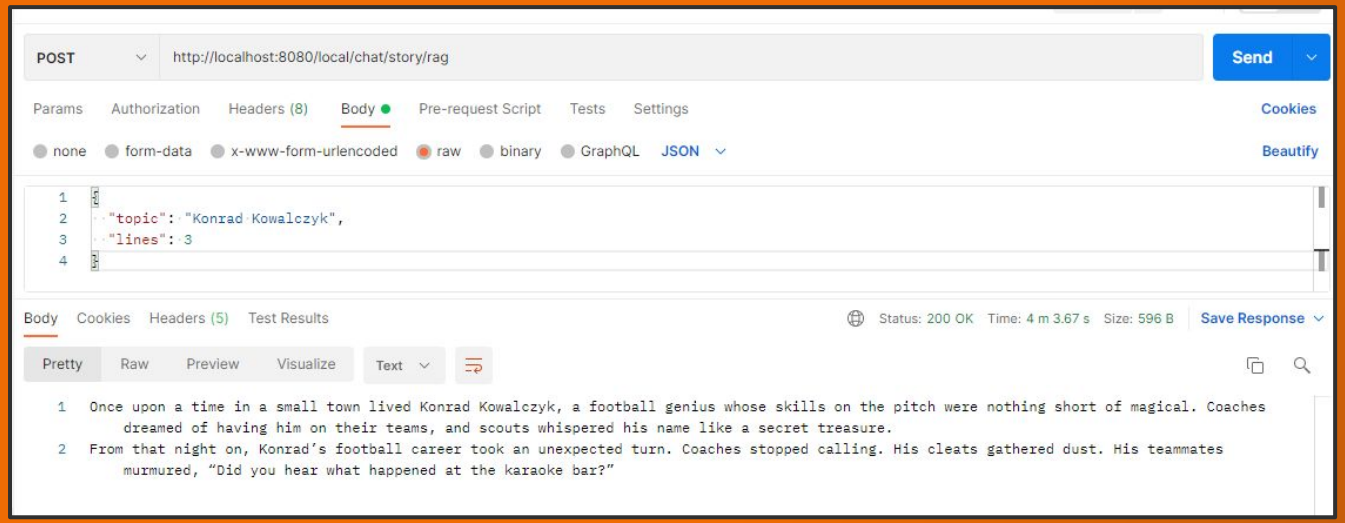
Bonus (RAG + Tools)

```
public LlamaArtistRAGService(@Qualifier("llamaModel") OllamaChatModel llamaChatModel,
                             ArtistToolComponent artistTool,
                             @Qualifier("story") RetrievalAugmentor augmentorStory) {
    this.assistant = AiServices.builder(ArtistAssistant.class)
            .chatModel(llamaChatModel)
            .tools(artistTool)
            .retrievalAugmentor(augmentorStory)
            .build();
}
```

Example (request)

```
}, {
  "role" : "tool",
  "content" : "generated-stories/story_0_20251029_220918.txt"
} ]
```

Not a perfect response, but the story was generated ✅



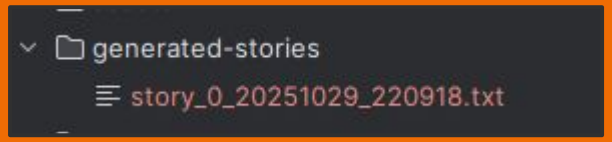
Example (logs)


```
"model" : "llama3.1:8b",
"messages" : [ {
  "role" : "system",
  "content" : "You are a professional author writing in the style of J.R.R. Tolkien. Your stories should reflect epic fantasy themes, deep world-building, and lyrical language."
}, {
  "role" : "user",
  "content" : "Write a story about Konrad Kowalczyk. The story should be 3 lines long and your response should only include the filename.\nThen save this story to a text file on disk.\n\n\nAnswer using the following information:\ncontent: Once upon a time in a small town lived Konrad Kowalczyk, a
   football genius whose skills on the pitch were nothing short of magical. Coaches dreamed of having him on their teams, and scouts whispered his name like a secret treasure.\nfile_name: history.txt\nindex: 0\nauthor: Konrad Kowalczyk\ncreated_at: 2025-01-01\n\ncontent: From that night on,
   Konrad's football career took an unexpected turn. Coaches stopped calling. His cleats gathered dust. His teammates murmured, "Did you hear what happened at the karaoke bar?"\nfile_name: history.txt\nindex: 6\nauthor: Konrad Kowalczyk\ncreated_at: 2025-01-01"
} ],
"options" : {
  "stop" : [ ]
```



```
"tools" : [ {
  "type" : "function",
  "function" : {
    "name" : "saveStoryToFile",
    "description" : "Save the given story to a file on disk and return name of the story",
    "parameters" : {
      "type" : "object",
      "properties" : {
        "story" : {
          "type" : "string"
        }
      },
      "required" : [ "story" ]
    }
  }
} ]
```

Example (result)




```
Once upon a time in a small town lived Konrad Kowalczyk, a football genius whose skills on the pitch were nothing short of magical.
Coaches dreamed of having him on their teams, and scouts whispered his name like a secret treasure.
From that night on, Konrad's football career took an unexpected turn.
Coaches stopped calling.
His cleats gathered dust.
His teammates murmured, "Did you hear what happened at the karaoke bar?"
```


```
2025-10-29T22:10:44.060+01:00  INFO 21128 --- [ai] [nio-8080-exec-2] d.l.http.client.log.LoggingHttpClient    : HTTP response:
- status code: 200
- headers: [content-length: 733], [content-type: application/json; charset=utf-8], [date: Wed, 29 Oct 2025 21:10:44 GMT]
- body: {"model":"llama3.1:8b","created_at":"2025-10-29T21:10:44.058873228Z","message":{"role":"assistant","content":"Once upon a time in a small town lived Konrad Kowalczyk, a football genius whose skills on the pitch were nothing short of magical. Coaches dreamed of having him on their teams, and scouts whispered his name like a secret treasure.\nFrom that night on, Konrad's football career took an unexpected turn. Coaches stopped calling. His cleats gathered dust. His teammates murmured, "Did you hear what happened at the karaoke bar?"**"},"done_reason":"stop","done":true,
"total_duration":24777352119,"load_duration":38193987,"prompt_eval_count":433,"prompt_eval_duration":272719577,"eval_count":95,"eval_duration":24459449767}
```

Model Context Protocol (MCP)

Model Context Protocol (MCP) is a standard that enables Large Language Models (LLMs) to communicate with external MCP compliant servers. These servers can provide and execute tools accessible via HTTP or stdio protocols.

MCP = Tools in other places



In a nutshell 😄

Example: MCP Example

Questions?
Thanks for your time!

Repo

